



LAPORAN PRAKTIKUM

TUGAS 1: RESTFUL API MURNI DENGAN EXPRESS.JS

Nama: Muhammad Ilham Fajri

NIM: 2428240082

Kelas: SI5B

Nomor Topik: 20

Nama Topik: Asuransi

Resource: Polis (/insurance-policies)

PROGRAM STUDI SISTEM INFORMASI
UNIVERSITAS MULTI DATA PALEMBANG
TAHUN AKADEMIK 2026/2027

1. Tujuan Praktikum

- Membangun RESTful API murni menggunakan Node.js dan Express.js untuk mengelola resource polis asuransi.
- Menerapkan operasi CRUD menggunakan method HTTP GET, POST, PUT, dan DELETE dengan parameter route serta query string sesuai ketentuan RESTful API.
- Menerapkan validasi field wajib, pilihan nilai pada field jenis, serta penggunaan HTTP status code 200, 201, 400, dan 404 pada response JSON.
- Melakukan pengujian seluruh endpoint, mengunggah source code ke repository GitHub, dan melakukan deployment API ke Vercel.

2. Dasar Teori

2.1 REST dan Resource

REST (Representational State Transfer) merupakan pendekatan dalam perancangan layanan web yang memanfaatkan resource dan method HTTP untuk melakukan operasi terhadap data. Dalam praktikum ini, resource yang digunakan adalah polis asuransi dengan endpoint utama /insurance-policies.

2.2 Method HTTP

GET digunakan untuk mengambil data, POST digunakan untuk menambahkan data baru, PUT digunakan untuk mengganti data berdasarkan identitas resource, sedangkan DELETE digunakan untuk menghapus data. Keenam endpoint yang diwajibkan pada tugas mencakup GET seluruh data, GET berdasarkan id, POST, PUT, DELETE, dan GET dengan query-string filter.

2.3 HTTP Status Code

Status code digunakan untuk menunjukkan hasil pemrosesan request. Status 200 menunjukkan request berhasil, 201 menunjukkan data berhasil dibuat, 400 menunjukkan request tidak valid, dan 404 menunjukkan resource atau endpoint tidak ditemukan.

2.4 JSON

JSON (JavaScript Object Notation) digunakan sebagai format pertukaran data pada API. Pada tugas ini seluruh response harus berbentuk JSON. Response GET mengembalikan

objek atau array secara langsung, sedangkan response POST, PUT, DELETE, dan error menggunakan struktur status, message, dan data.

2.5 Express.js

Express.js adalah framework Node.js yang digunakan untuk membuat server dan routing HTTP. Express menyediakan `express.json()` untuk membaca request body berformat JSON serta mendukung route parameter dan query string yang digunakan pada API praktikum ini.

3. Alat dan Bahan

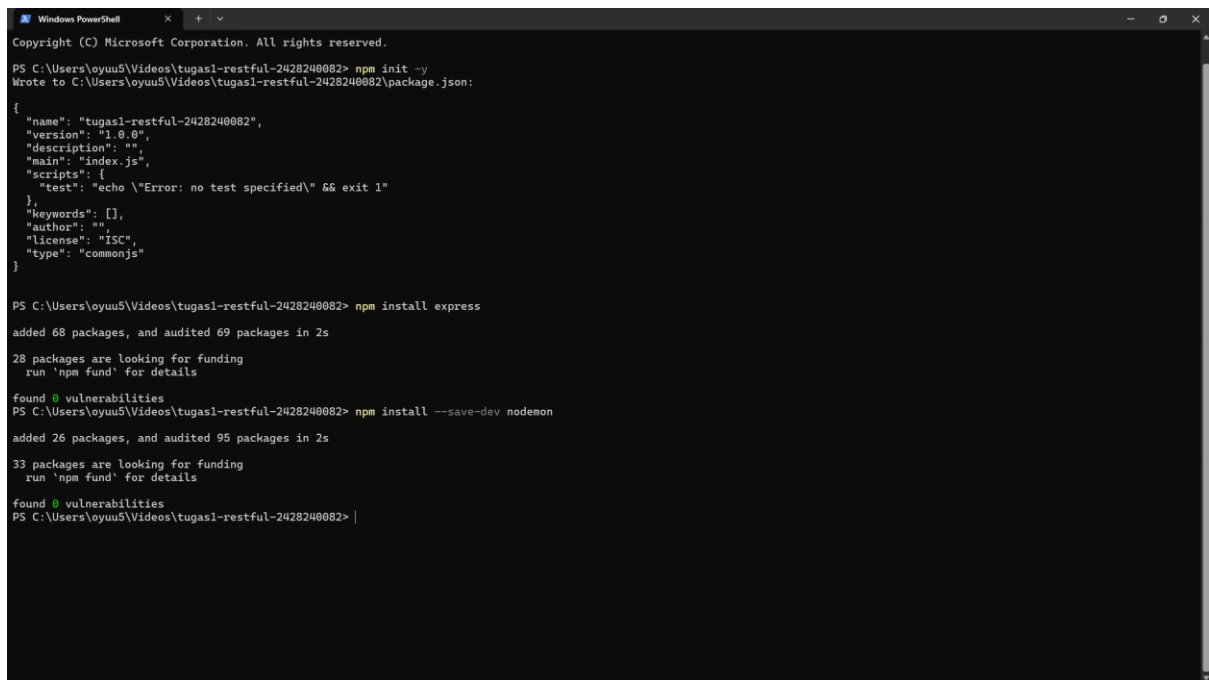
- Node.js LTS dan npm.
- Express.js sebagai framework REST API dan nodemon sebagai dependency pengembangan.
- Visual Studio Code sebagai editor kode.
- Postman, Thunder Client, atau Insomnia untuk pengujian endpoint.
- Git dan GitHub untuk version control dan repository source code.
- Vercel untuk deployment RESTful API.

4. Langkah Praktikum

4.1 Inisialisasi Project

Project dibuat menggunakan Node.js. Package project diinisialisasi dengan npm, kemudian Express.js dipasang sebagai dependency dan nodemon sebagai devDependency.

```
npm init -y
npm install express
npm install --save-dev nodemon
```



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\oyuu5\Videos\tugas1-restful-2428240082> npm init -y
Wrote to C:\Users\oyuu5\Videos\tugas1-restful-2428240082\package.json:

{
  "name": "tugas1-restful-2428240082",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \\Error: no test specified\\ && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

PS C:\Users\oyuu5\Videos\tugas1-restful-2428240082> npm install express
added 68 packages, and audited 69 packages in 2s

28 packages are looking for funding
  run 'npm fund' for details

found 0 vulnerabilities
PS C:\Users\oyuu5\Videos\tugas1-restful-2428240082> npm install --save-dev nodemon
added 26 packages, and audited 95 packages in 2s

33 packages are looking for funding
  run 'npm fund' for details

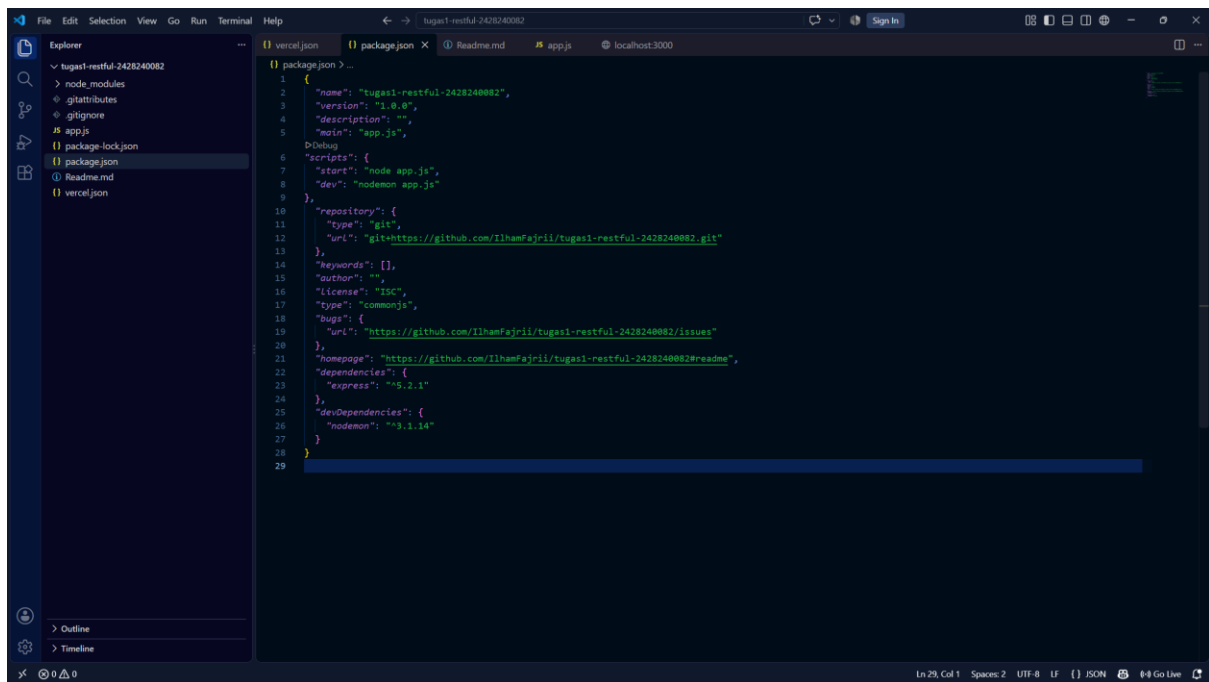
found 0 vulnerabilities
PS C:\Users\oyuu5\Videos\tugas1-restful-2428240082> |
```

Gambar — Inisialisasi project dan hasil instalasi dependency

4.2 Konfigurasi package.json

File package.json disesuaikan agar app.js menjadi entry point dan tersedia script start serta dev.

```
{
  "main": "app.js",
  "scripts": {
    "start": "node app.js",
    "dev": "nodemon app.js"
  }
}
```



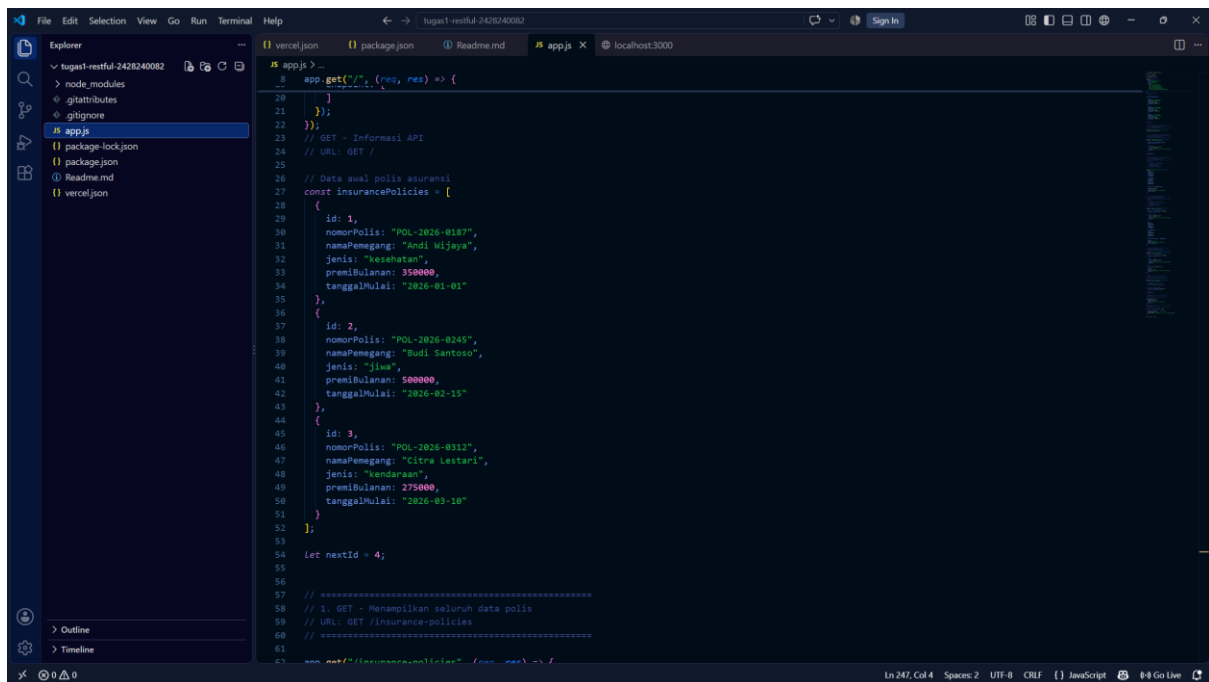
Gambar — package.json

4.3 Data Awal Resource Polis

Resource yang digunakan adalah polis asuransi. Berdasarkan topik nomor 20, endpoint yang digunakan adalah /insurance-policies dengan filter berdasarkan field jenis. Field wajib terdiri dari nomorPolis, namaPemegang, jenis, premiBulanan, dan tanggalMulai. Nilai jenis yang diperbolehkan adalah kesehatan, jiwa, dan kendaraan.

```
let insurancePolicies = [
  {
    id: 1,
    nomorPolis: "POL-2026-0187",
    namaPemegang: "Andi Wijaya",
    jenis: "kesehatan",
    premiBulanan: 350000,
    tanggalMulai: "2026-01-01"
  },
  {
    id: 2,
    nomorPolis: "POL-2026-0245",
    namaPemegang: "Budi Santoso",
    jenis: "jiwa",
    premiBulanan: 500000,
    tanggalMulai: "2026-02-15"
  },
  {
    id: 3,
    nomorPolis: "POL-2026-0312",
    namaPemegang: "Citra Lestari",
    jenis: "kendaraan",
    premiBulanan: 275000,
    tanggalMulai: "2026-03-10"
  }
];
```

```
let nextId = 4;
```



Gambar — Data awal insurancePolicies di app.js

4.4 Konfigurasi Express dan Endpoint Utama

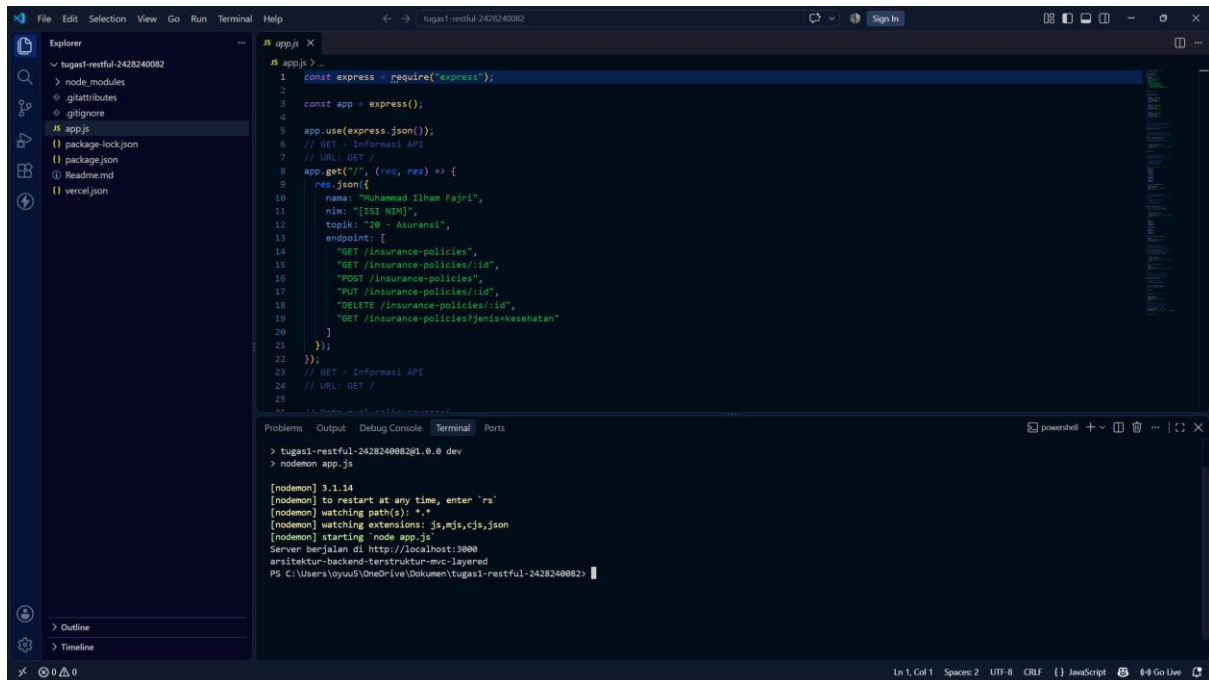
```

const express = require("express");
const app = express();

app.use(express.json());

app.get("/", (req, res) => {
  res.json({
    nama: "Muhammad Ilham Fajri",
    nim: "[ISI NIM]",
    topik: "20 - Asuransi",
    endpoints: [
      "GET /insurance-policies",
      "GET /insurance-policies/:id",
      "POST /insurance-policies",
      "PUT /insurance-policies/:id",
      "DELETE /insurance-policies/:id",
      "GET /insurance-policies?jenis=kesehatan"
    ]
  });
});

```



Gambar — Route GET / dan middleware express.json()

4.5 Route GET Seluruh Data dan Filter

GET /insurance-policies digunakan untuk mengambil seluruh data. Jika query jenis diberikan, data difilter menggunakan req.query dan response tetap berupa array.

```
app.get("/insurance-policies", (req, res) => {
  const { jenis } = req.query;

  // Jika tidak menggunakan filter
  if (!jenis) {
    return res.json(insurancePolicies);
  }

  // Filter berdasarkan jenis
  const hasil = insurancePolicies.filter(
    (item) => item.jenis === jenis
  );

  res.json(hasil);
});
```

```
// =====  
// FILTER  
// GET /insurance-policies?jenis=kesehatan  
// =====  
  
app.get("/insurance-policies", (req, res) => {  
  const { jenis } = req.query;  
  
  // Jika tidak menggunakan filter  
  if (!jenis) {  
    return res.json(insurancePolicies);  
  }  
  
  // Filter berdasarkan jenis  
  const hasil = insurancePolicies.filter(  
    (item) => item.jenis === jenis  
  );  
  
  res.json(hasil);  
});
```

Gambar — Route GET /insurance-policies dan filter jenis

4.6 Route GET Berdasarkan ID

```
app.get("/insurance-policies/:id", (req, res) => {  
  const id = Number(req.params.id);  
  const policy = insurancePolicies.find(item => item.id === id);  
  
  if (!policy) {  
    return res.status(404).json({  
      status: "error",  
      message: `Data dengan id ${id} tidak ditemukan`,  
      data: null  
    });  
  }  
  
  res.json(policy);  
});
```

```

5
6
7 // =====
8 // 2. GET - Menampilkan satu data berdasarkan ID
9 // URL: GET /insurance-policies/:id
10 // =====
11
12 app.get("/insurance-policies/:id", (req, res) => {
13   const id = Number(req.params.id);
14
15   const policy = insurancePolicies.find(
16     (item) => item.id === id
17   );
18
19   if (!policy) {
20     return res.status(404).json({
21       status: "error",
22       message: `Data dengan id ${id} tidak ditemukan`,
23       data: null
24     });
25   }
26
27   res.json(policy);
28 });
29

```

Gambar — Route GET /insurance-policies/:id

4.7 Route POST

POST digunakan untuk menambahkan polis baru. ID dibuat oleh server sehingga tidak dikirim dalam request body. Field wajib divalidasi sebelum data dimasukkan ke array.

```

app.post("/insurance-policies", (req, res) => {
  const {
    nomorPolis, namaPemegang, jenis,
    premiBulanan, tanggalMulai
  } = req.body;

  const jenisValid = ["kesehatan", "jiwa", "kendaraan"];

  if (!nomorPolis)
    return res.status(400).json({
      status: "error",
      message: "Field nomorPolis wajib diisi",
      data: null
    });

  if (!namaPemegang)
    return res.status(400).json({
      status: "error",
      message: "Field namaPemegang wajib diisi",
      data: null
    });

  if (!jenis)

```

```
        return res.status(400).json({
            status: "error",
            message: "Field jenis wajib diisi",
            data: null
        });

    if (!jenisValid.includes(jenis))
        return res.status(400).json({
            status: "error",
            message: "Field jenis harus kesehatan, jiwa, atau kendaraan",
            data: null
        });

    if (premiBulanan === undefined || premiBulanan === null || premiBulanan ===
    "")
        return res.status(400).json({
            status: "error",
            message: "Field premiBulanan wajib diisi",
            data: null
        });

    if (!tanggalMulai)
        return res.status(400).json({
            status: "error",
            message: "Field tanggalMulai wajib diisi",
            data: null
        });

    const newPolicy = {
        id: nextId++,
        nomorPolis,
        namaPemegang,
        jenis,
        premiBulanan: Number(premiBulanan),
        tanggalMulai
    };

    insurancePolicies.push(newPolicy);

    res.status(201).json({
        status: "success",
        message: "Data berhasil ditambahkan",
        data: newPolicy
    });
});
```

```
// =====
// 3. POST - Menambahkan data polis baru
// URL: POST /insurance-policies
// =====
// Contoh body:
// {
//   "nomorPolis": "POL-2026-0401",
//   "namaPemegang": "Dedi Saputra",
//   "jenis": "kesehatan",
//   "premiBulanan": 400000,
//   "tanggalMulai": "2026-04-01"
// }

app.post("/insurance-policies", (req, res) => {
  const {
    nomorPolis,
    namaPemegang,
    jenis,
    premiBulanan,
    tanggalMulai
  } = req.body;

  const newPolicy = {
    id: nextId++,
    nomorPolis,
    namaPemegang,
    jenis,
    premiBulanan,
    tanggalMulai
  };

  insurancePolicies.push(newPolicy);

  res.status(201).json({
    status: "success",
    message: "Data berhasil ditambahkan",
    data: newPolicy
  });
});
```

Gambar — Route POST dan validasi field

4.8 Route PUT

PUT digunakan untuk mengganti seluruh data polis pada id tertentu. Karena bersifat replacement, seluruh field wajib harus dikirim.

```
app.put("/insurance-policies/:id", (req, res) => {
  const id = Number(req.params.id);
  const index = insurancePolicies.findIndex(item => item.id === id);

  if (index === -1) {
    return res.status(404).json({
      status: "error",
      message: `Data dengan id ${id} tidak ditemukan`,
      data: null
    });
  }

  const {
    nomorPolis, namaPemegang, jenis,
```

```
    premiBulanan, tanggalMulai
  } = req.body;

  const jenisValid = ["kesehatan", "jiwa", "kendaraan"];

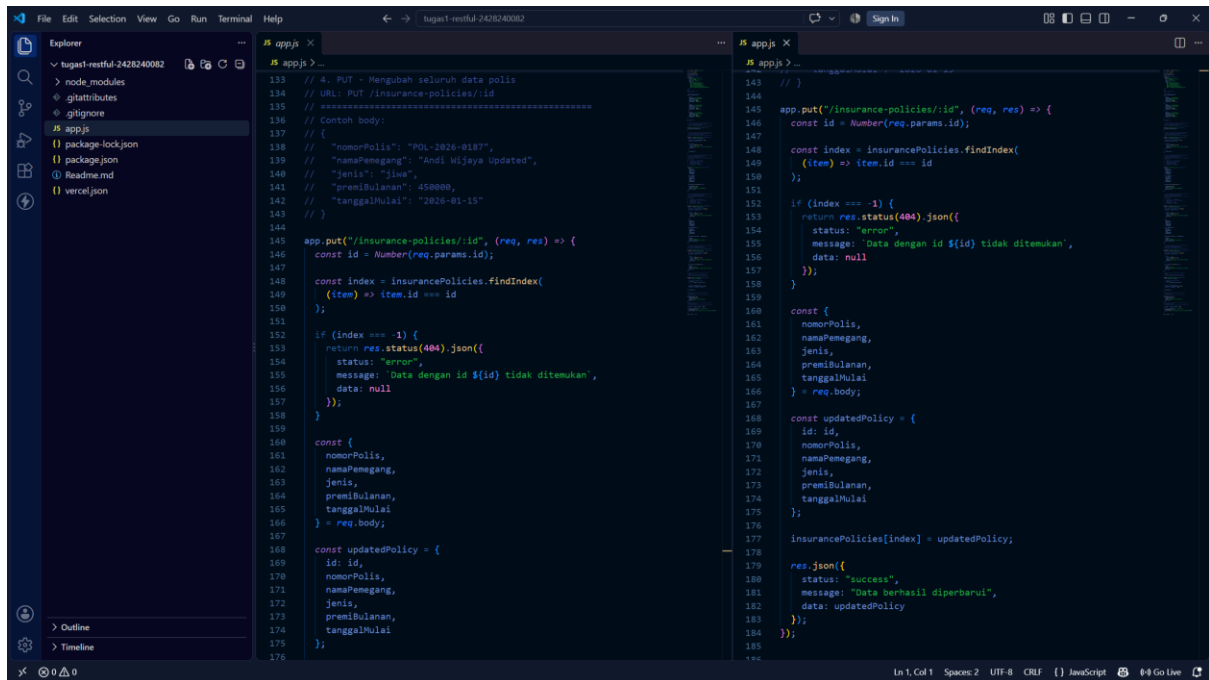
  if (!nomorPolis || !namaPemegang || !jenis ||
    premiBulanan === undefined || premiBulanan === null ||
    premiBulanan === "" || !tanggalMulai) {
    return res.status(400).json({
      status: "error",
      message: "Semua field wajib harus diisi",
      data: null
    });
  }

  if (!jenisValid.includes(jenis)) {
    return res.status(400).json({
      status: "error",
      message: "Field jenis harus kesehatan, jiwa, atau kendaraan",
      data: null
    });
  }

  const updatedPolicy = {
    id,
    nomorPolis,
    namaPemegang,
    jenis,
    premiBulanan: Number(premiBulanan),
    tanggalMulai
  };

  insurancePolicies[index] = updatedPolicy;

  res.json({
    status: "success",
    message: "Data berhasil diperbarui",
    data: updatedPolicy
  });
});
```



Gambar — Route PUT dan validasi replacement

4.9 Route DELETE

```
app.delete("/insurance-policies/:id", (req, res) => {
  const id = Number(req.params.id);
  const index = insurancePolicies.findIndex(item => item.id === id);

  if (index === -1) {
    return res.status(404).json({
      status: "error",
      message: `Data dengan id ${id} tidak ditemukan`,
      data: null
    });
  }

  insurancePolicies.splice(index, 1);

  res.json({
    status: "success",
    message: `Data dengan id ${id} berhasil dihapus`,
    data: null
  });
});
```

```
86
87 // =====
88 // 5. DELETE - Menghapus data polis
89 // URL: DELETE /insurance-policies/:id
90 // =====
91
92 app.delete("/insurance-policies/:id", (req, res) => {
93   const id = Number(req.params.id);
94
95   const index = insurancePolicies.findIndex(
96     (item) => item.id === id
97   );
98
99   if (index === -1) {
100     return res.status(404).json({
101       status: "error",
102       message: `Data dengan id ${id} tidak ditemukan`,
103       data: null
104     });
105   }
106
107   insurancePolicies.splice(index, 1);
108
109   res.json({
110     status: "success",
111     message: `Data dengan id ${id} berhasil dihapus`,
112     data: null
113   });
114 });
115
```

Gambar — Route DELETE

4.10 Middleware 404 dan Menjalankan Server

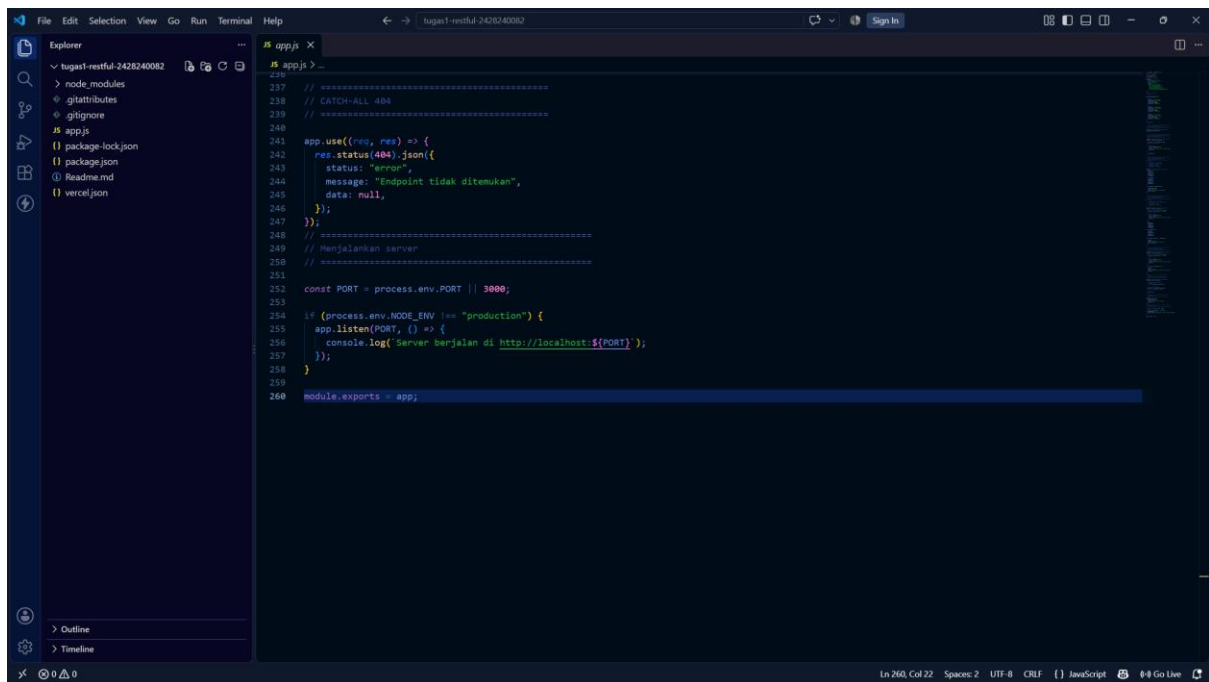
```
app.use((req, res) => {
  res.status(404).json({
    status: "error",
    message: "Endpoint tidak ditemukan",
    data: null
  });
});

const PORT = process.env.PORT || 3000;

if (process.env.NODE_ENV !== "production") {
  app.listen(PORT, () => {
    console.log(`Server berjalan di http://localhost:${PORT}`);
  });
}

module.exports = app;
```

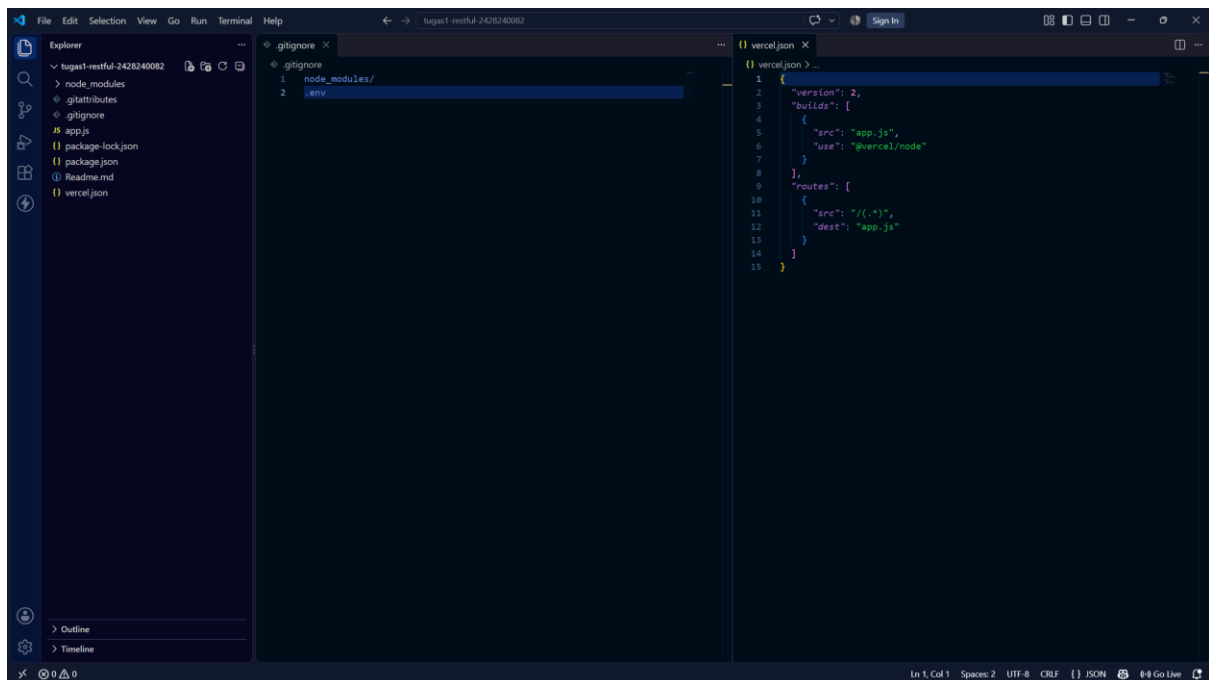
Middleware 404 digunakan untuk menangani endpoint yang tidak tersedia. Untuk deployment Vercel, app diekspor melalui module.exports dan app.listen hanya dijalankan ketika bukan production.



Gambar — Middleware 404 dan terminal server

4.11 .gitignore dan vercel.json

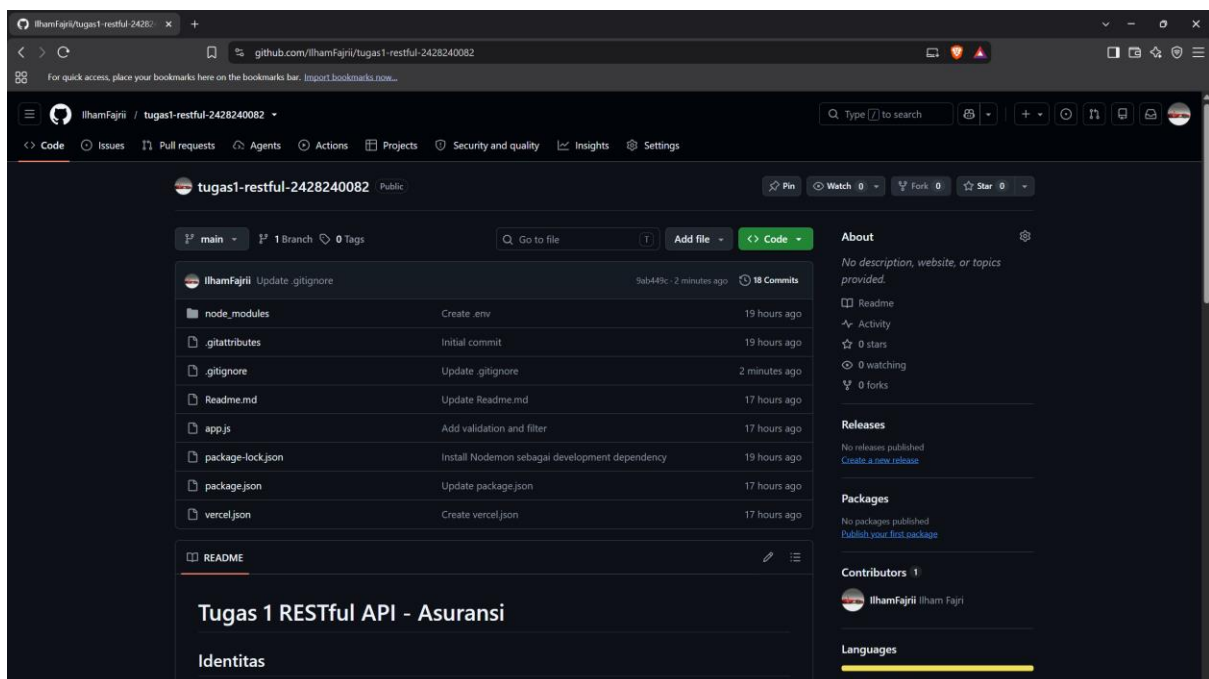
```
.gitignore
node_modules/
.env
vercel.json
{
  "version": 2,
  "builds": [
    { "src": "app.js", "use": "@vercel/node" }
  ],
  "routes": [
    { "src": "/*", "dest": "app.js" }
  ]
}
```

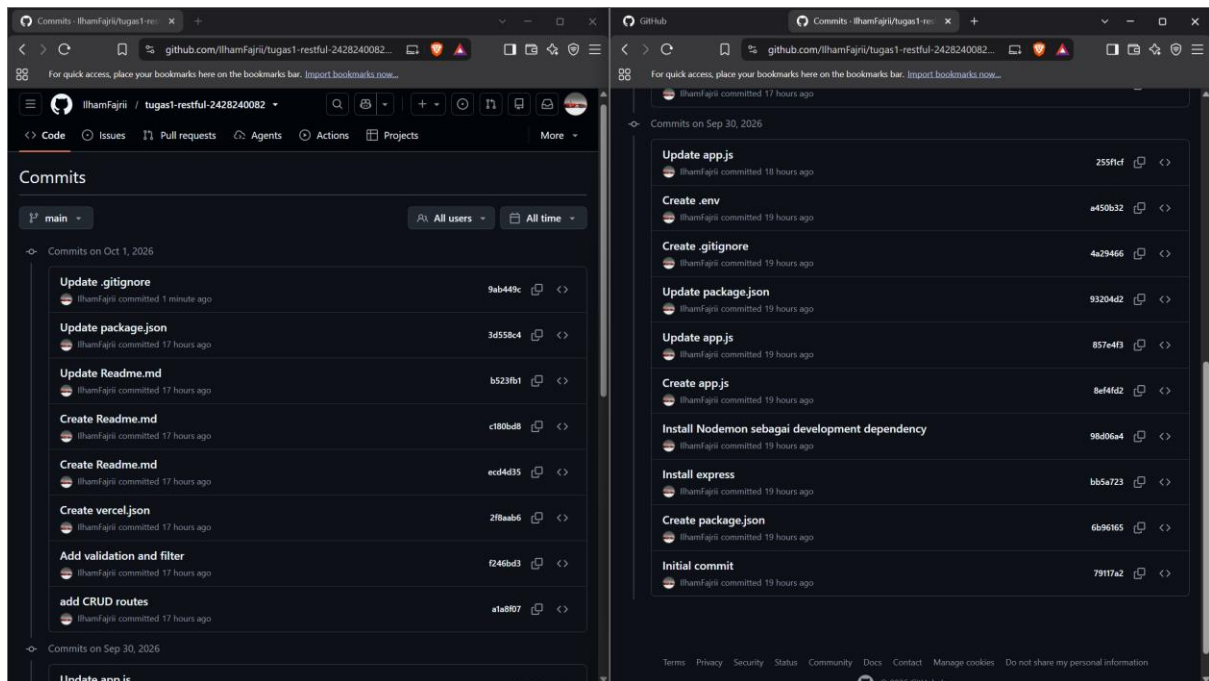


Gambar — File `.gitignore` dan `vercel.json`

4.12 Push GitHub

Source code diunggah ke repository GitHub publik dengan nama `tugas1-restful-2428240082`. Repository minimal memiliki lima staged commits sesuai ketentuan tugas.

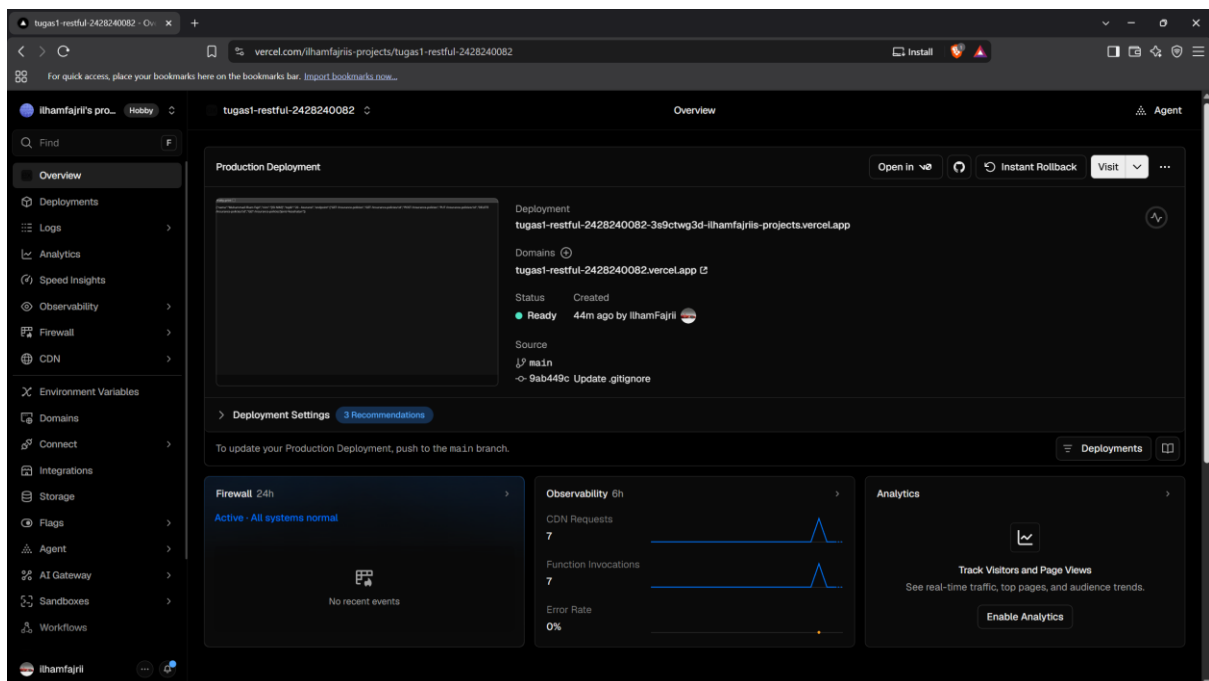




Gambar — Repository GitHub dan riwayat commit

4.13 Deploy ke Vercel

Repository GitHub dihubungkan ke Vercel dan dilakukan deployment. Setelah berhasil, seluruh endpoint diuji kembali menggunakan URL Vercel. Data bersifat in-memory sehingga dapat kembali ke data awal ketika instance serverless berubah.

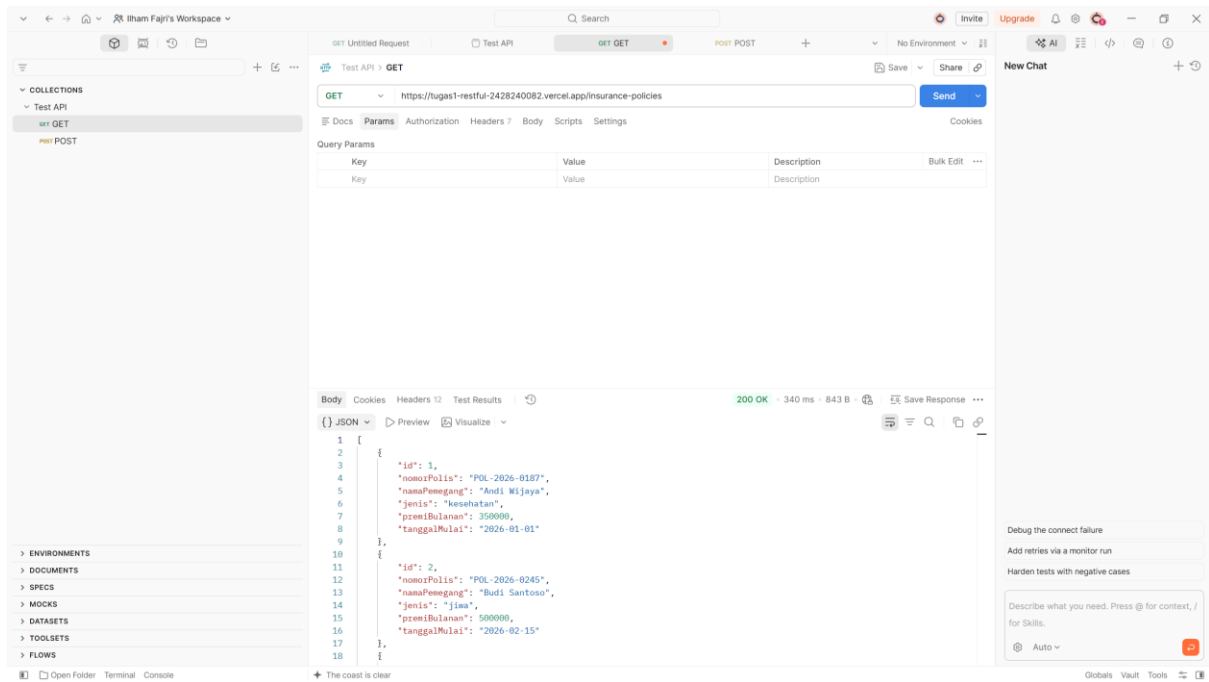


Gambar — Dashboard Vercel dan URL deployment

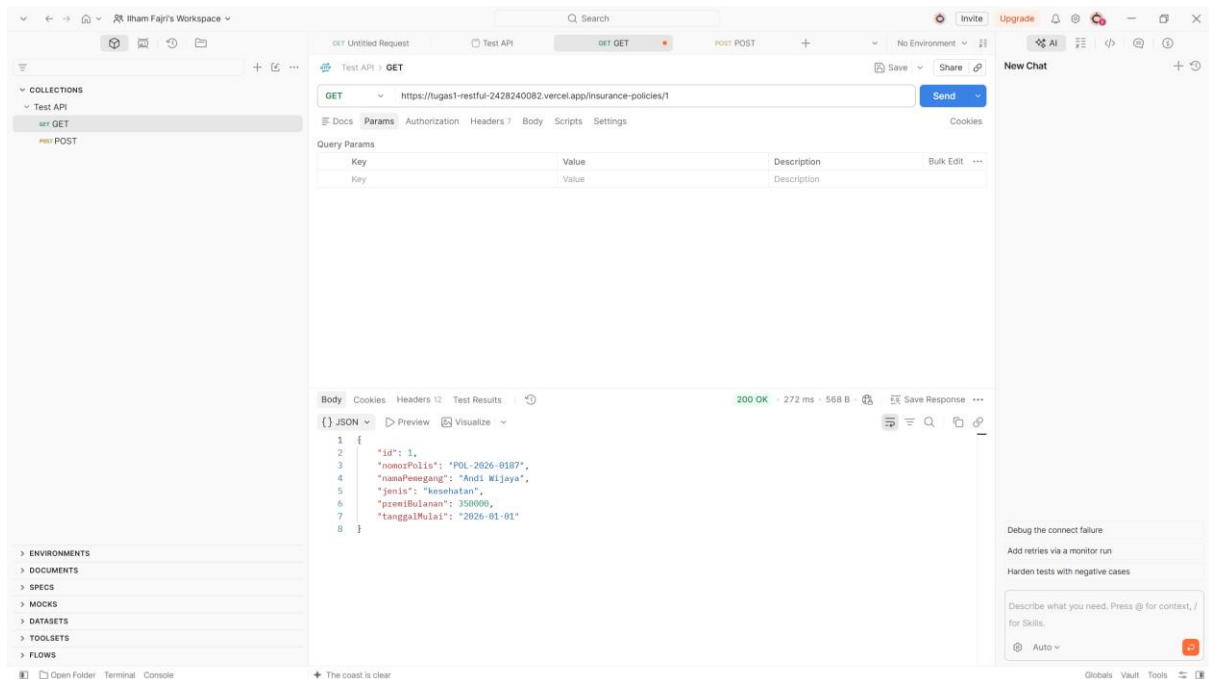
5. Hasil Pengujian

Pengujian dilakukan terhadap URL deployment Vercel. Kolom status hasil dan keterangan berikut disiapkan untuk diisi berdasarkan hasil pengujian aktual setelah screenshot request dan response dilampirkan.

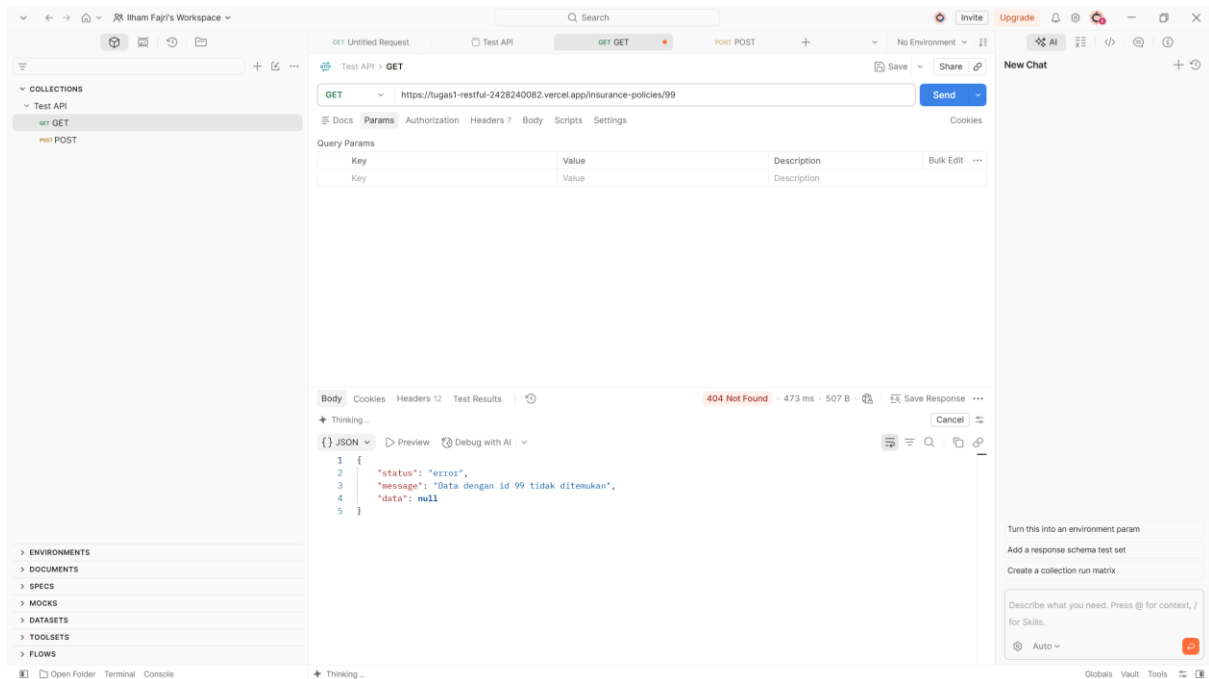
No	Method	Endpoint	Data request	Status diharapkan	Status hasil / Keterangan
1	GET	/insurance-policies	—	200	Sesuai
2	GET	/insurance-policies/1	—	200	Sesuai
3	GET	/insurance-policies/99	—	404	Sesuai
4	GET	/insurance-policies?jenis=kesehatan	—	200	Sesuai
5	POST	/insurance-policies	Body lengkap: nomorPolis, namaPemegang, jenis, premiBulanan, tanggalMulai	201	Sesuai
6	POST	/insurance-policies	Field wajib dikosongkan	400	Sesuai
7	PUT	/insurance-policies/1	Body lengkap	200	Sesuai
8	PUT	/insurance-policies/99	Body lengkap	404	Sesuai
9	DELETE	/insurance-policies/1	—	200	Sesuai
10	DELETE	/insurance-policies/99	—	404	Sesuai



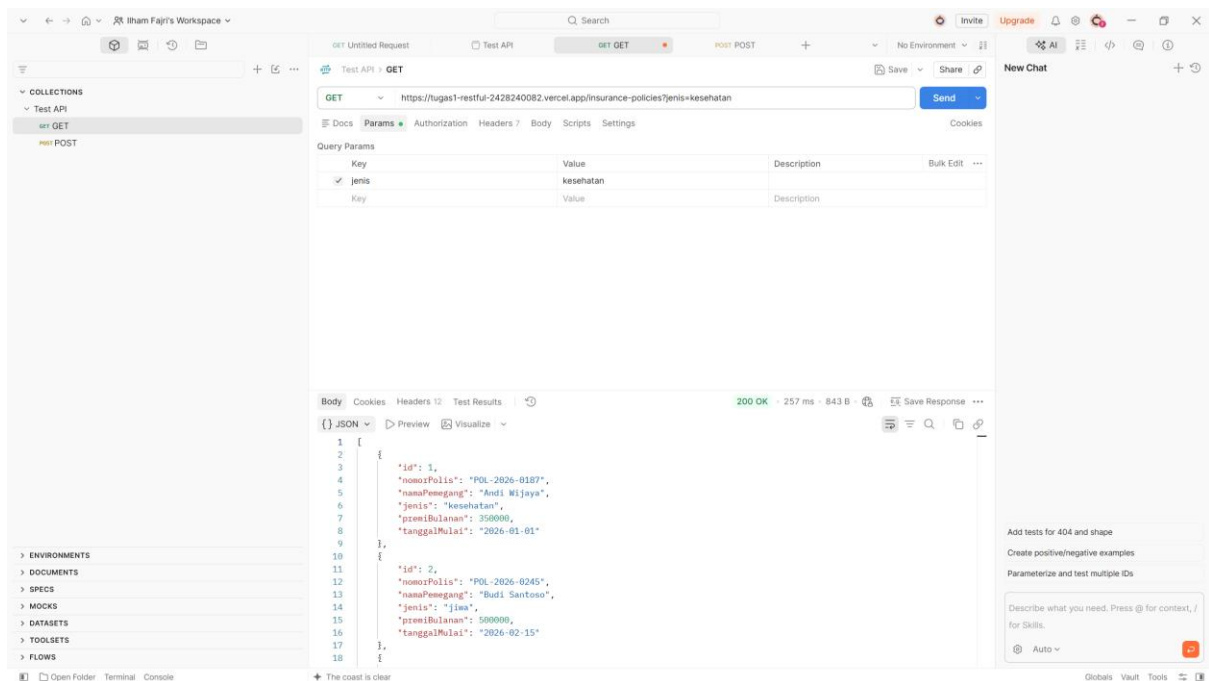
Gambar — GET seluruh data



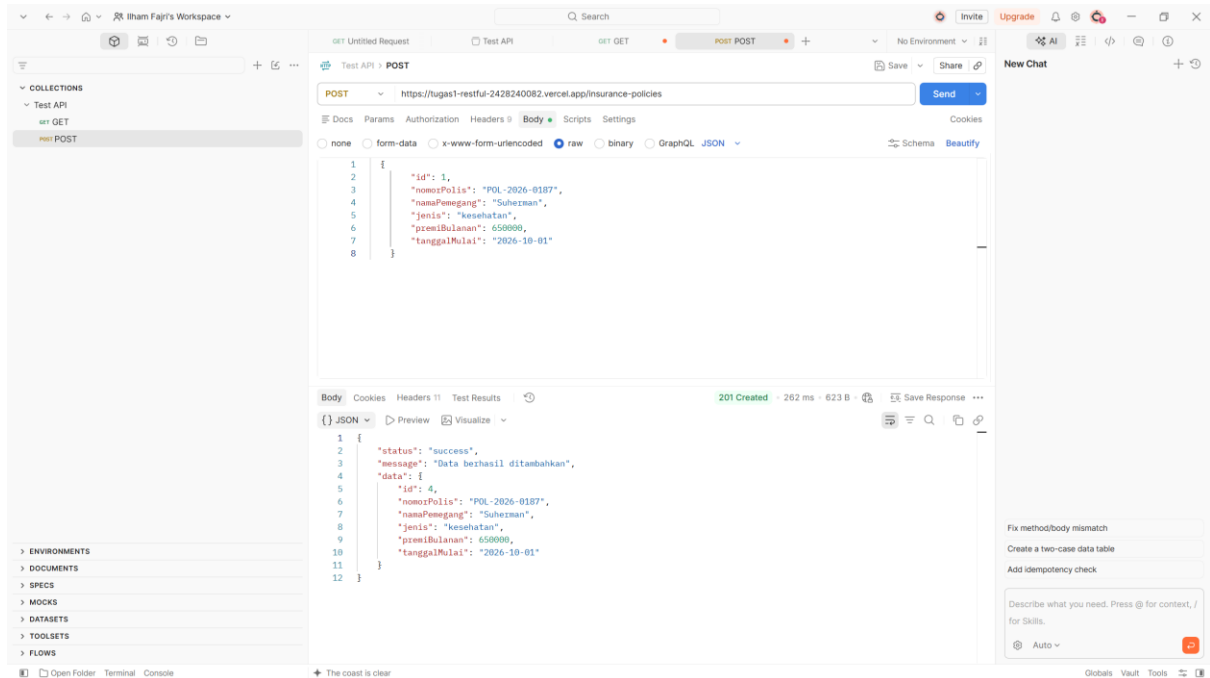
Gambar — GET berdasarkan id



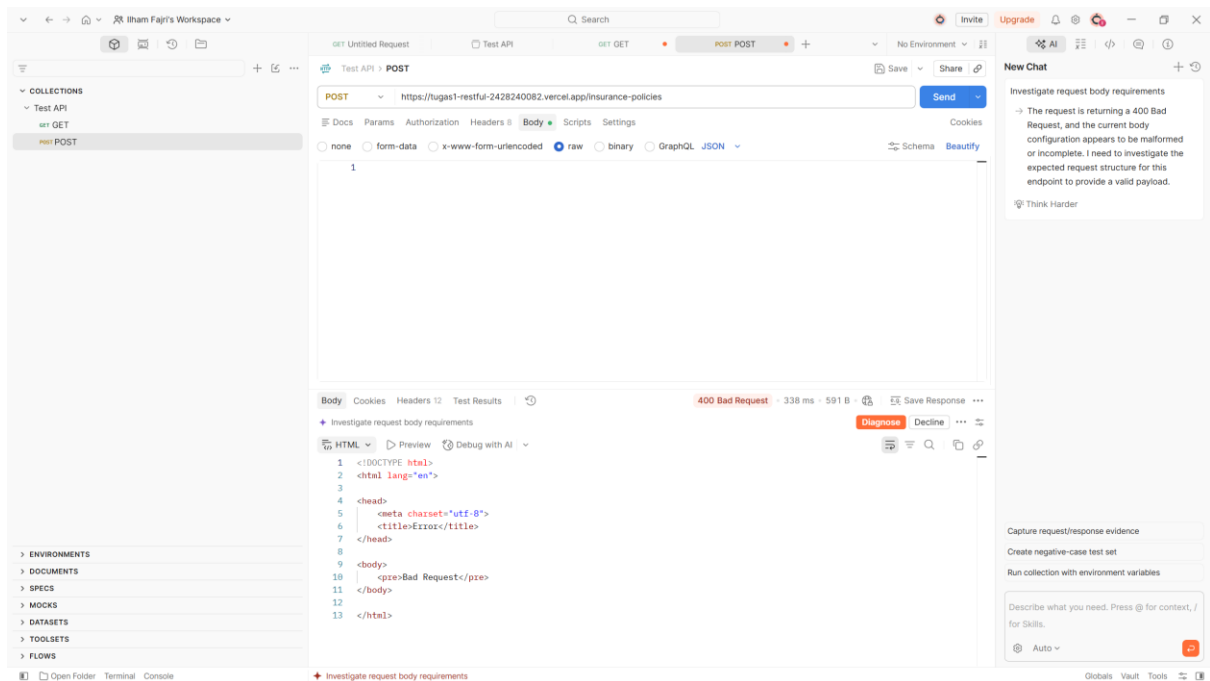
Gambar — GET id tidak ditemukan (404)



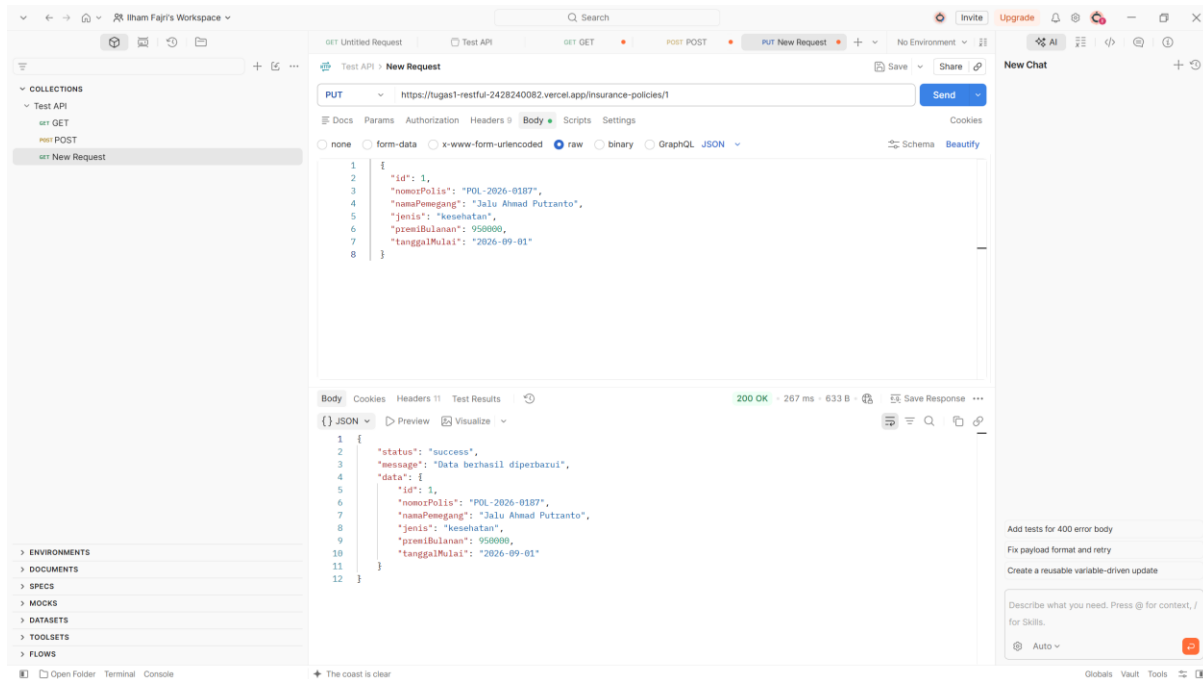
Gambar — GET filter jenis



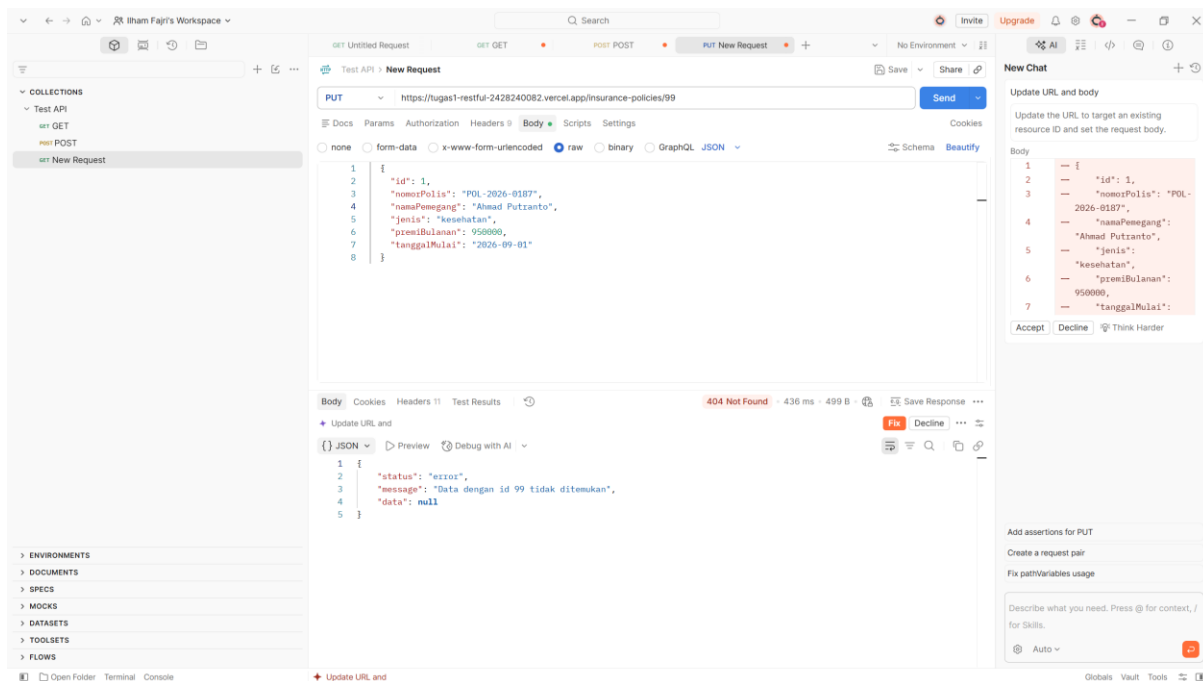
Gambar — POST body lengkap (201)



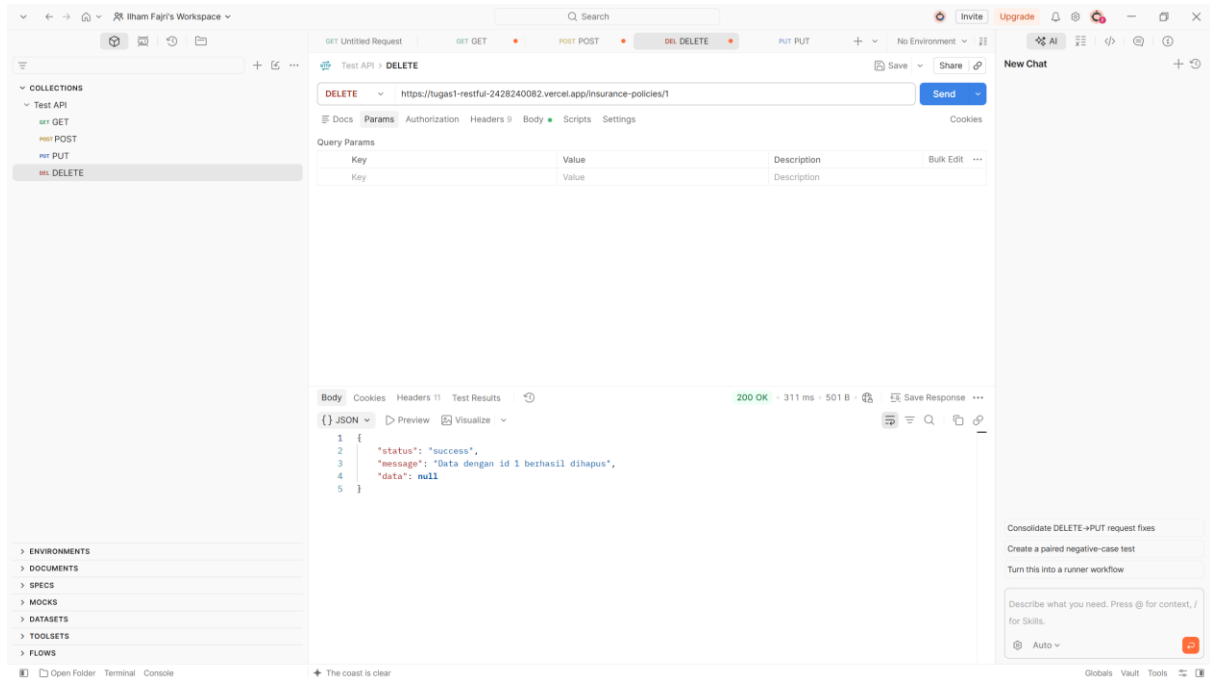
Gambar — POST field wajib kosong (400)



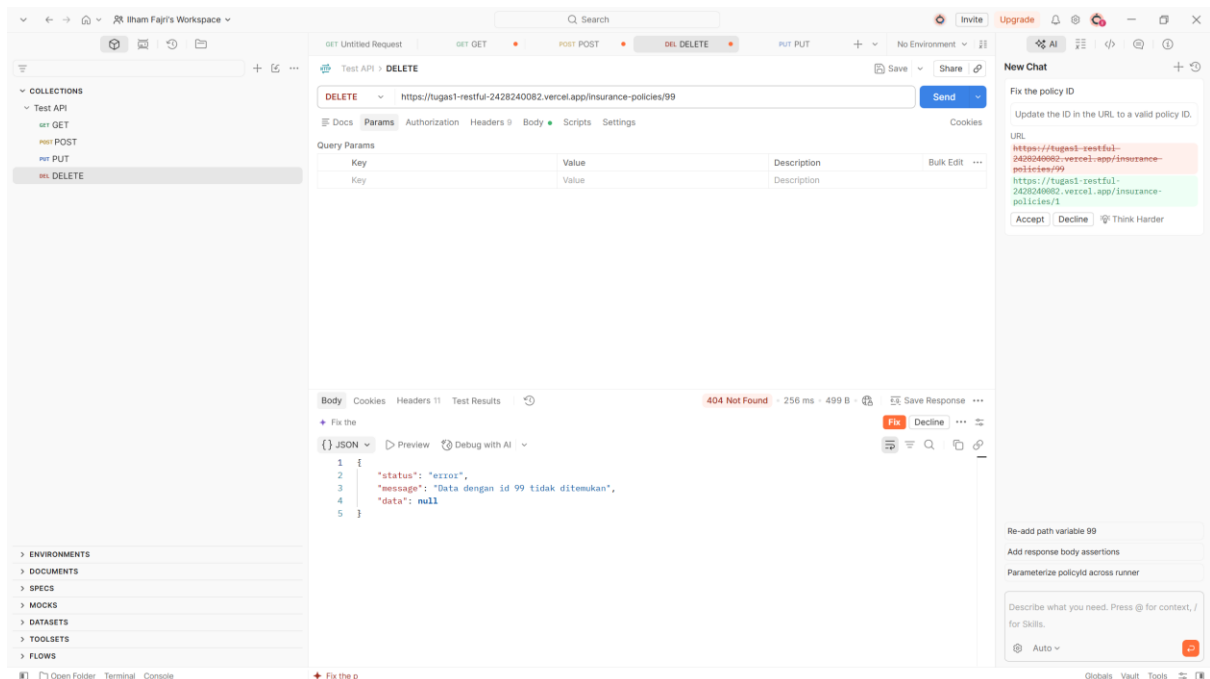
Gambar — PUT body lengkap (200)



Gambar — PUT id tidak ditemukan (404)



Gambar — DELETE id tersedia (200)



Gambar — DELETE id tidak ditemukan (404)

6. Pembahasan

6.1 Pemilihan Method dan Status Code

Method HTTP dipilih berdasarkan operasi yang dilakukan terhadap resource polis. GET digunakan untuk membaca data, POST untuk membuat data baru, PUT untuk mengganti data secara keseluruhan, dan DELETE untuk menghapus data. Status 200 digunakan pada operasi yang berhasil, 201 pada pembuatan data baru, 400 ketika input tidak memenuhi validasi, dan 404 ketika data atau endpoint tidak ditemukan.

6.2 Validasi Data

Validasi diperlukan agar field wajib tidak kosong dan nilai jenis hanya berasal dari pilihan kesehatan, jiwa, atau kendaraan. Pada PUT, seluruh field wajib dikirim karena operasi tersebut mengganti data secara keseluruhan.

6.3 Kendala dan Penyelesaian

Kendala yang dapat muncul selama pengerjaan antara lain kesalahan penulisan endpoint, format JSON request yang tidak sesuai, validasi field, konfigurasi Vercel, dan perbedaan perilaku data in-memory setelah deployment. Penyelesaiannya dilakukan dengan memeriksa kembali route, request body, status code, konfigurasi vercel.json, serta melakukan pengujian ulang pada setiap endpoint.

Catatan: bagian ini perlu disesuaikan dengan kendala yang benar-benar dialami saat implementasi dan deployment.

7. Kesimpulan

Praktikum ini menghasilkan rancangan RESTful API untuk resource polis asuransi menggunakan Express.js. API menyediakan operasi membaca, menambah, memperbarui, menghapus, dan memfilter data melalui endpoint /insurance-policies. Implementasi juga menerapkan validasi field wajib, status code HTTP, response JSON, middleware 404, repository GitHub, serta deployment ke Vercel. Dengan pengujian terhadap seluruh endpoint, API dapat dievaluasi berdasarkan kesesuaian method, endpoint, request, response, dan status code dengan ketentuan tugas.

8. Lampiran

A. Repository GitHub

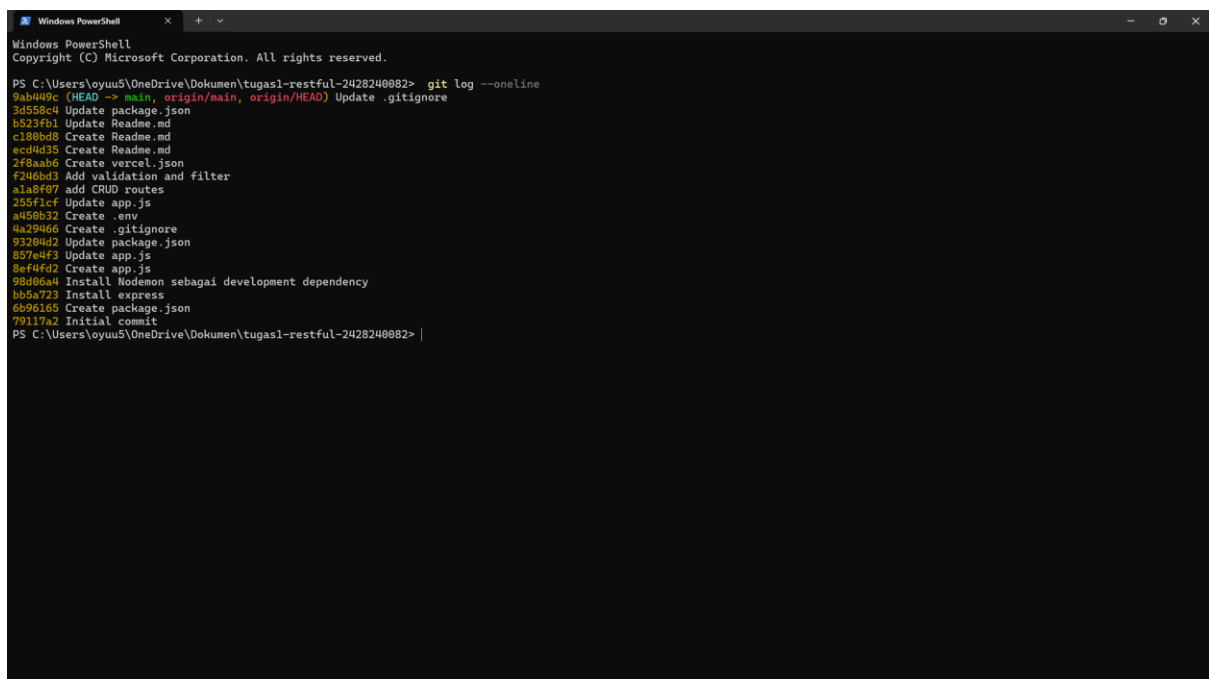
[<https://github.com/IlhamFajrii/tugas1-restful-2428240082.git>]

B. Deployment Vercel

[tugas1-restful-2428240082-3s9ctwg3d-ilhamfajriis-projects.vercel.app]

C. Riwayat Commit

Lampirkan screenshot hasil perintah git log --oneline yang menunjukkan minimal lima commit.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\oyuu5\OneDrive\Dokumen\tugas1-restful-2428240082> git log --oneline
9ab449c (HEAD -> main, origin/main, origin/HEAD) Update .gitignore
3d558c4 Update package.json
b523fb1 Update Readme.md
c188bd8 Create Readme.md
ec9dd15 Create Readme.md
2f8aabb Create vercel.json
f246bd3 Add validation and filter
a1a8f97 add CRUD routes
255f1cf Update app.js
a458b32 Create .env
4a294d6 Create .gitignore
93284d2 Update package.json
857e4f3 Update app.js
8ef4fd2 Create app.js
98d86a4 Install Nodemon sebagai development dependency
bb5a723 Install express
6b9d165 Create package.json
79117a2 Initial commit
PS C:\Users\oyuu5\OneDrive\Dokumen\tugas1-restful-2428240082> |
```

Gambar — git log --oneline minimal 5 commit